

Synthesis Report

5,856	6	87.9%	97.1%	0.995
Total X-Ray Images	ML Models Trained	PCA Variance Retained	Best Accuracy (Split 1)	Best ROC-AUC (SVM Split 1)

Abstract

For this project we built a system that looks at chest X-ray images and tries to tell if a patient has pneumonia or not. We tested six different classifiers — Logistic Regression, Random Forest, MLP, SVM, KNN, and a CNN — and compared them across three different ways of splitting the data. The hardest part turned out to be the class imbalance: about 74% of the images are pneumonia cases, so every model we trained at first just predicted pneumonia for everything and got 0% on normal cases. Fixing that took more time than we expected. We also did a 3-class version at the end to separate bacterial vs viral pneumonia.

1. Project Overview

Goal

Given a chest X-ray image, predict whether the lungs look **NORMAL** (label 0) or show signs of **PNEUMONIA** (label 1). The dataset comes from Kaggle (Chest X-Ray Pneumonia) and is split across three separate folders.

Dataset Breakdown

Dataset	Role	Normal	Pneumonia	Total
Dataset 1 (Train)	Training	1,341	3,875	5,216
Dataset 2 (Val)	Validation / Tuning	8	8	16
Dataset 3 (Test)	Final Evaluation	234	390	624
TOTAL	—	1,583	4,273	5,856

The imbalance is pretty obvious — roughly 3 pneumonia images for every 1 normal one. This ended up being the main problem throughout the whole project. Models without any correction just learned to always say pneumonia and still got 74% accuracy, which looks fine on paper but is completely wrong in practice.

2. Preprocessing Pipeline

Every image goes through the same steps before reaching the model. The CNN is a slight exception since it works on 2D pixel grids directly, but the grayscale and resize steps still apply.

1	Grayscale (gray filter) We load each image and convert it to grayscale using PIL's <code>.convert('L')</code> . X-rays don't carry colour information so keeping RGB would just add noise.
2	Resize to 128x128 Images come from different scanners at different resolutions. We resize everything to 128x128 so the feature vector length stays consistent.
3	Flatten to 1D + Normalise 128x128 = 16,384 pixels. We call <code>.flatten()</code> to turn the 2D image into a 1D array, then divide by 255.0 to bring values into [0.0, 1.0].
4	StandardScaler Fit only on training data. Shifts each of the 16,384 features to mean=0, std=1. Without this, features with large pixel ranges would dominate the distance calculations.
5	PCA — 100 components Drops from 16,384 features down to 100. We keep 87.88% of the variance. This made SVM and KNN actually trainable — without PCA they were running for hours.

After preprocessing: X_train shape (5,216 x 100) | X_val (16 x 100) | X_test (624 x 100)

3. Data Splitting Strategies

Split 1 — Simple Train/Test

We take 80% of Dataset 1 for training (4,172 images) and leave 20% for testing (1,044 images). Stratified so the class ratio stays the same in both halves. Good for a quick check but the numbers tend to look better than reality.

Since both halves come from the same dataset, there's some risk the split just got lucky. Scores on Split 1 are generally the highest — don't read too much into them.

Split 2 — Train / Validation / Test

Dataset 1 trains the model, Dataset 2 (just 16 images) is used for hyperparameter tuning, and Dataset 3 (624 images) is the final held-out test. This is the most honest setup since the test images come from a completely separate source.

The validation set being only 16 images is a real problem. GridSearchCV on 16 images isn't very reliable, so the tuning step is a bit shaky.

Split 3 — 5-Fold Stratified CV

Dataset 1 gets split into 5 equal folds. Each time, 4 folds train and 1 fold tests, rotating through all 5. The reported score is the average. This gives a much more stable picture of how the model actually performs.

Takes longer to run but the results are the most trustworthy. We used this as the main comparison point between models.

4. Model Results

4.1 Logistic Regression

We started with logistic regression as a baseline. It's a linear model so we didn't expect miracles, but it actually did well on Split 1 and Split 3. Split 2 drops noticeably — probably because Dataset 3 comes from a different source than what the model trained on.

Split	Accuracy	F1	ROC-AUC
Split 1 (Train/Test)	0.9598	0.9728	0.9891
Split 2 (Val/Test)	0.7484	0.8310	0.8973
Split 3 (5-Fold CV)	0.9559	0.9704	0.9871

4.2 Random Forest

Random forest with 100 trees. We expected this to beat logistic regression but it actually came in slightly lower on most splits. On PCA-reduced data the trees tend to be less effective — the principal components don't have the same interpretability as raw features. Still solid results overall.

Split	Accuracy	F1	ROC-AUC
Split 1 (Train/Test)	0.9416	0.9617	0.9858
Split 2 (Val/Test)	0.7436	0.8287	0.9160
Split 3 (5-Fold CV)	0.9329	0.9562	0.9833

4.3 MLP Neural Network

MLP with two hidden layers (256 then 128 nodes). This was the first model where we saw real improvement on the normal class after adding class weights and rotating some normal images to balance the training set. The best sklearn model we got.

Split	Accuracy	F1	ROC-AUC
Split 1 (Train/Test)	0.9617	0.9741	0.9937
Split 2 (Val/Test)	0.7837	0.8512	0.9032
Split 3 (5-Fold CV)	0.9680	0.9785	0.9928

4.4 SVM — Support Vector Machine

SVM with RBF kernel gave us the most trouble to get right. The first version predicted 100% pneumonia — literally 0% normal recall. The fix was changing the GridSearchCV scoring from f1 to **balanced_accuracy** and setting gamma to 'scale' instead of a fixed small value. After that it became our best model on Split 1.

Split	Accuracy	F1	ROC-AUC	Normal %
Split 1 (Train/Test)	0.9713	0.9805	0.9952	97.01%
Split 2 (Val/Test)	0.7949	0.8562	0.9161	49.15%
Split 3 (5-Fold CV)	0.9682	0.9784	0.9952	~96%

The problem with f1 scoring is that it only looks at the pneumonia class (the positive class). A model that says pneumonia for everything gets a high f1 score. Balanced accuracy forces the model to do well on both classes, which is what we actually want.

4.5 KNN — K-Nearest Neighbours

KNN works by finding the K closest training points in PCA space and going with majority vote. We swept K from 1 to 20 and picked the best one (K=9). It's surprisingly competitive — simpler than SVM but gets close on most splits. Normal class recall was also decent compared to some of the other models.

Split	Accuracy	F1	ROC-AUC	Normal %
Split 1 (K=9)	0.9617	0.9745	0.9809	89.93%
Split 2 (K=9)	0.7676	0.8419	0.8741	~60%
Split 3 (K=9)	0.9515	0.9677	0.9838	~88%

4.6 CNN — Convolutional Neural Network

The CNN was the most involved to set up. Three convolutional blocks with batch normalisation and dropout, trained directly on 64x64 grayscale images (no PCA). We added class weights to stop it ignoring normal images, and tuned the prediction threshold to 0.7 to push normal recall up a bit more.

Configuration	Accuracy	F1	ROC-AUC	Normal %	Pneumonia %
Split 2, thresh=0.5	0.8542	0.8936	0.9499	64.53%	97.95%
Split 2, thresh=0.7	0.8574	0.8947	0.9499	67.09%	96.92%

Architecture: Input(64x64x1) → Conv2D(32) → BN → MaxPool → Dropout(0.25) → Conv2D(64) → BN → MaxPool → Dropout(0.25) → Conv2D(128) → BN → MaxPool → Dropout(0.4) → Dense(256) → Dropout(0.5) → Dense(1, sigmoid)

5. Final Model Comparison

Split 2 is the one that matters most — it uses a completely separate test set that the models never saw during training. Split 1 and Split 3 scores are higher but they're less honest since the data comes from the same source as training.

Model	Split	Accuracy	F1	ROC-AUC
Logistic Regression	Split 1	0.9598	0.9728	0.9891
Logistic Regression	Split 2	0.7484	0.8310	0.8973
Logistic Regression	Split 3 CV	0.9559	0.9704	0.9871
Random Forest	Split 1	0.9416	0.9617	0.9858
Random Forest	Split 2	0.7436	0.8287	0.9160
Random Forest	Split 3 CV	0.9329	0.9562	0.9833
MLP Neural Net	Split 1	0.9617	0.9741	0.9937
MLP Neural Net	Split 2	0.7837	0.8512	0.9032
MLP Neural Net	Split 3 CV	0.9680	0.9785	0.9928
SVM (RBF, balanced)	Split 1	0.9713	0.9805	0.9952
SVM (RBF, balanced)	Split 2	0.7949	0.8562	0.9161
SVM (RBF, balanced)	Split 3 CV	0.9682	0.9784	0.9952
KNN (K=9)	Split 1	0.9617	0.9745	0.9809
KNN (K=9)	Split 2	0.7676	0.8419	0.8741
KNN (K=9)	Split 3 CV	0.9515	0.9677	0.9838
CNN (thresh=0.7)	Split 2	0.8574	0.8947	0.9499

Best Overall — SVM (Split 1): Accuracy 0.9713 | F1 0.9805 | ROC-AUC 0.9952 | Normal 97.01% | Pneumonia 97.16%

6. What Went Wrong and How We Fixed It

Normal class accuracy stuck at 0%

What happened	The dataset has 1,341 normal images vs 3,875 pneumonia. Every model we trained initially just predicted pneumonia for everything. Accuracy looked decent (74%) but normal recall was literally 0%. Took us a while to even realise the problem.
What we did	We rotated normal training images 4 times to get more copies (from 1,341 to ~6,700). Added <code>class_weight='balanced'</code> to SVM, KNN, and CNN. For SVM specifically, changed GridSearchCV scoring to <code>balanced_accuracy</code> . Threshold tuned to 0.7 on CNN. After all this, normal accuracy on SVM reached 97%.

Raw images too big for SVM and KNN

What happened	Each 128x128 image flattened is 16,384 features. Training SVM on 5,000 images with 16k features is very slow — we left it running and it was still going after 30 minutes.
What we did	PCA down to 100 components. Keeps 87.88% of the variance but cuts training time to seconds. The 99.4% feature reduction made both SVM and KNN completely usable.

SVM GridSearchCV chose a broken model

What happened	When we used <code>scoring='f1'</code> in GridSearchCV, the best model it found had 0% normal recall. The issue is f1 only measures pneumonia (the positive class) — so predicting everything as pneumonia still scores well. The grid also picked <code>gamma=0.001</code> which made the RBF kernel almost flat.
What we did	Changed scoring to <code>balanced_accuracy</code> , which cares about both classes equally. Also switched <code>gamma</code> to 'scale' so sklearn auto-adjusts it based on the data. Normal accuracy went from 0% to 97% just from this change.

Validation set too small to be useful

What happened	Dataset 2 only has 16 images — 8 normal, 8 pneumonia. Running GridSearchCV on 16 samples gives basically random results. One bad prediction swings the score by 12.5%.
What we did	We kept Dataset 2 only for the final held-out test, not for tuning. For actual hyperparameter decisions we relied on Split 3 (5-fold CV on Dataset 1) which is much more stable.

TensorFlow not compatible with Python 3.14

What happened	The project virtual environment runs Python 3.14 but tensorflow-macos has no wheel for Python 3.14. Every install attempt just failed silently or errored.
What we did	Installed TensorFlow on the system Python 3.11 instead (<code>/opt/homebrew/bin/python3.11</code>). Also had to remove tensorflow-metal because it kept throwing a 'platform already registered: METAL' conflict at startup.

7. Bonus — 3-Class: Normal / Bacteria / Virus

Once the binary version was working we tried going one step further — instead of just normal vs pneumonia, can we tell the difference between bacterial and viral pneumonia too? The filenames in the PNEUMONIA folder include either 'bacteria' or 'virus', so we used that to split them into two separate classes. Normal stays class 0, bacteria is 1, virus is 2.

How many images per class

Dataset	NORMAL	BACTERIA	VIRUS	Total
Train (Dataset 1)	1,341	2,530	1,345	5,216
Test (Dataset 3)	234	242	148	624

Virus is the hardest class by far. Only 148 test images, and bacterial vs viral pneumonia looks very similar on an X-ray — even doctors sometimes struggle. The models all confused virus with bacteria regularly.

Results across all models

Model	Split	Accuracy	Macro F1	Weighted F1
SVM (OvR, RBF)	Split 1	0.8151	0.8108	0.8138
SVM (OvR, RBF)	Split 2 (Official)	0.6939	0.6734	0.6867
SVM (OvR, RBF)	Split 3 CV	0.7991	0.7938	0.7981
KNN (K=18)	Split 1	0.7835	0.7656	0.7746
KNN (K=18)	Split 2 (Official)	0.7019	0.6789	0.6927
KNN (K=18)	Split 3 CV	0.7697	0.7469	0.7596
CNN (3-class softmax)	Split 2 (Official)	0.7228	0.7015	0.7186

SVM per class — Split 1 (best result)

Class	Precision	Recall	F1-Score	Support
NORMAL	0.9664	0.9628	0.9646	269
BACTERIA	0.8107	0.8379	0.8241	506
VIRUS	0.6640	0.6245	0.6437	269
Macro avg	0.8137	0.8084	0.8108	1044

CNN per class — Split 2 (official test)

Class	Precision	Recall	F1-Score	Support
NORMAL	0.9909	0.4658	0.6337	234
BACTERIA	0.8201	0.9421	0.8769	242
VIRUS	0.4831	0.7703	0.5938	148
Macro avg	0.7647	0.7261	0.7015	624

The CNN had a weird split — precision on normal was 99% but recall was only 46%. That means when it said normal it was almost always right, but it missed more than half of the actual normal cases. The softmax pushes it to commit to bacteria a lot because that's the biggest class. SVM with OvR handled all three classes more evenly and ended up with the better macro F1 overall.

Changes made for 3-class

```
CNN output: Dense(3, softmax) instead of Dense(1, sigmoid)
CNN loss: sparse_categorical_crossentropy
SVM: decision_function_shape='ovr' (one model per class vs the rest)
KNN best K: 18 (re-swept from scratch on 3-class data)
```

8. What We Learned

Overall the project went well but it took longer than expected, mostly because of the class imbalance and getting the evaluation metric right. A few things stood out:

- SVM ended up being the best model — 97% on both classes in Split 1 and 0.9952 ROC-AUC. The trick was the scoring metric in GridSearchCV, not the model itself.
- Getting 0% normal accuracy was a confusing bug to track down. The model looked fine by overall accuracy but was completely wrong on half the possible outputs.
- PCA was non-negotiable for SVM and KNN. Without it, those two were unusable on a standard laptop — training just never finished.
- Cross-validation (Split 3) gave us the most consistent numbers. Split 1 varied a lot depending on the random seed; Split 3 averaged that out.
- CNN was the only model that learns spatial patterns — lung texture, infiltrate shapes. The others just see 100 PCA numbers. That's probably why it handles normal cases differently and why it gets confused differently too.
- Split 2 scores being 0.15-0.20 lower than Split 1/3 shows how much performance can drop when you test on a genuinely different dataset. That gap is probably closer to what real-world performance would look like.
- 3-class was harder as expected. Virus recall was weak across all models — the visual difference between bacterial and viral pneumonia is subtle even to experts.

Files submitted

- zoidberg2_pneumonia.ipynb — full notebook with all models, splits, and outputs
- zoidberg2_pneumonia.html — exported notebook in HTML
- synthesis_report.pdf — this report
- models/svm_best.joblib — SVM binary model
- models/knn_best.joblib — KNN binary model
- models/cnn_best.keras — CNN binary model weights
- models/svm3_best.joblib, knn3_best.joblib, cnn3_best.keras — 3-class models
- outputs/figures/ — all plots (ROC, confusion matrices, training curves, K sweep)

Zoidberg 2.0 — Epitech MSc IT Machine Learning — May 2026